

Краткая документация к математической библиотеке

dumb_math

Шелонин Арсений, Б01-411

 @Senchchch
 shelonin.ak@phystech.edu
 github.com/71frukt

Проект представляет собой интерфейсную библиотеку, объединяющую набор статических математических библиотек, реализующих модули различного функционала. Модуль **Benchmarking** используется многими другими модулями для реализации performance тестирования.

Модуль **Random** представляет собой набор инструментов для генерации случайных чисел, генерации распределений, а также тестирования внешних генераторов случайных чисел. Среди генераторов **engines** есть генераторы, поддерживающие параллелизацию по потокам. Такие генераторы должны поддерживать концепт, определённый в модуле параллелизатора по потокам **rng_parallelizer**

```
template <typename T>
concept RngParallelizable = requires(T a, uint64_t steps) {
    { a.skipahead(steps) } -> std::same_as<void>;
    { T::dimension()      } -> std::same_as<uint32_t>;
};
```

Функция, проводящая параллелизацию по ядрам процессора имеет следующий интерфейс:

```
template <concepts::RngParallelizable RngT, typename TaskFunc>
requires requires(TaskFunc task, RngT& rng, uint64_t count) {
    { task(rng, count) } -> std::default_initializable;
}

auto RngParallelRun(uint64_t total_elements, uint32_t skipahead_step, uint32_t seed,
                    TaskFunc task, int rt_priority = 0, std::vector<int> target_cores = {});
```

Модуль логарифма представляет собой отдельную самодостаточную (в том смысле, что не использует внешний инструментарий для внутренних расчётов) библиотеку, реализующую функцию расчёта логарифма:

```
template <std::floating_point T>
T ln(T x);
```

Для бенчмарков также используется модуль **Benchmarking**.

Модуль **Matrix** реализует класс Матрицы и различные методы перемножения двух матриц, имеющие следующий интерфейс формата

```
static void MatrMul(const Matrix& m1, const Matrix& m2, Matrix& m_dest);
```

Методы реализованы разными способами, начиная от примитивного и заканчивая векторизованным AVX2 блочным перемножением матриц.

Модуль, связанный с финансами, реализует функцию расчёта опциона европейского рынка несколькими способами: аналитически, методом Монте-Карло, и методом Монте-Карло с векторизацией AVX2 и параллелизацией по ядрам. Параллелизованная функция использует функционал векторного генератора **minstd** и параллелизатора по ядрам.

Обоснование архитектурных решений. Выбран многоуровневый стиль архитектуры (Layered Style). Данное решение изолирует высокоуровневую финансовую логику от низкоуровневых оптимизаций (AVX2 реализация генератора случайных чисел, распределение по потокам). Применение C++ Concepts для модуля **rng_parallelizer** позволяет достичь статического полиморфизма, исключая накладные расходы виртуальных вызовов в критических циклах расчетов Монте-Карло. Зависимости от модуля **Benchmarking** строго инкапсулированы в подмодули тестирования, что исключает транзитивные зависимости в релизных сборках модулей **Matrix** и **Finance**.

Интеграция со внешними системами. Библиотека **dumb_math** спроектирована как встраиваемый вычислительный движок. Она предназначена для интеграции в следующие типы систем:

- HFT и Algo-trading платформы: в качестве ядра для быстрого пересчета «греков» и цен опционов в реальном времени.
- Системы управления рисками: для массовой симуляции сценариев методом Монте-Карло при оценке портфеля.
- Аналитические Python-окружения: возможна интеграция в качестве C++ бэкенда через **pybind11** для ускорения расчетов.

Аппаратное обеспечение и среда выполнения

- Архитектура: Серверные или десктопные процессоры архитектуры x86-64.
- Наборы инструкций: Обязательная аппаратная поддержка расширений AVX2 и FMA (процессоры Intel Haswell / AMD Zen и новее).
- Среда выполнения: Многоядерные системы под управлением ОС Linux. Требуется поддержка POSIX Threads (**pthread**) для управления аффинностью потоков и планировщиком операционной системы.

В разработке модулей применяются следующие технические ограничения и стандарты:

- **Стандарт языка:** ISO C++20.

- **ОС и многопоточность:** Модуль `rng_parallelizer` использует библиотеку POSIX Threads (`pthread`) для управления потоками и привязки к физическим ядрам процессора. В связи с этим сборка и эксплуатация библиотеки поддерживаются исключительно в среде ОС Linux.
- **Векторизация и архитектура:** Компиляция вычислительно емких слоев (`Random`, `Matrix`, `Finance`) требует поддержки наборов инструкций AVX2 и FMA (флаги компиляции `-mavx2`, `-mfma`, `-O3`).
- **Внешние зависимости:** Требуется обязательная линковка с библиотекой потоков (`-lpthread`) и векторной математической библиотекой `glibc` (`-lmvec`) для расчета трансцендентных функций в опционах.

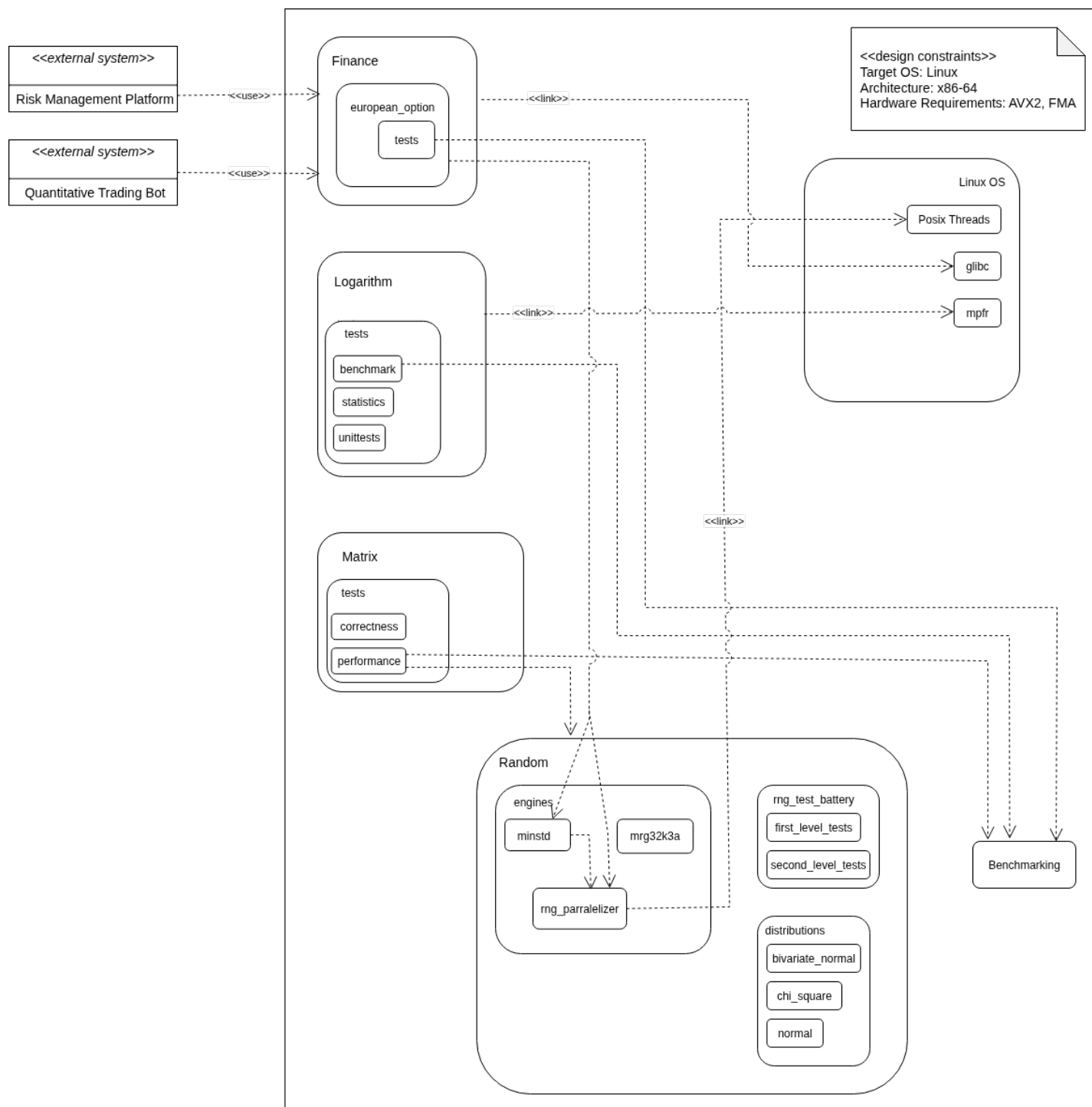


Рис. 1: Архитектурная схема системы `dumb_math`